

A self-organizing AI engineering company

Give it a mission — it designs its own team of specialist AI agents, builds in parallel, reviews itself, and ships working code.

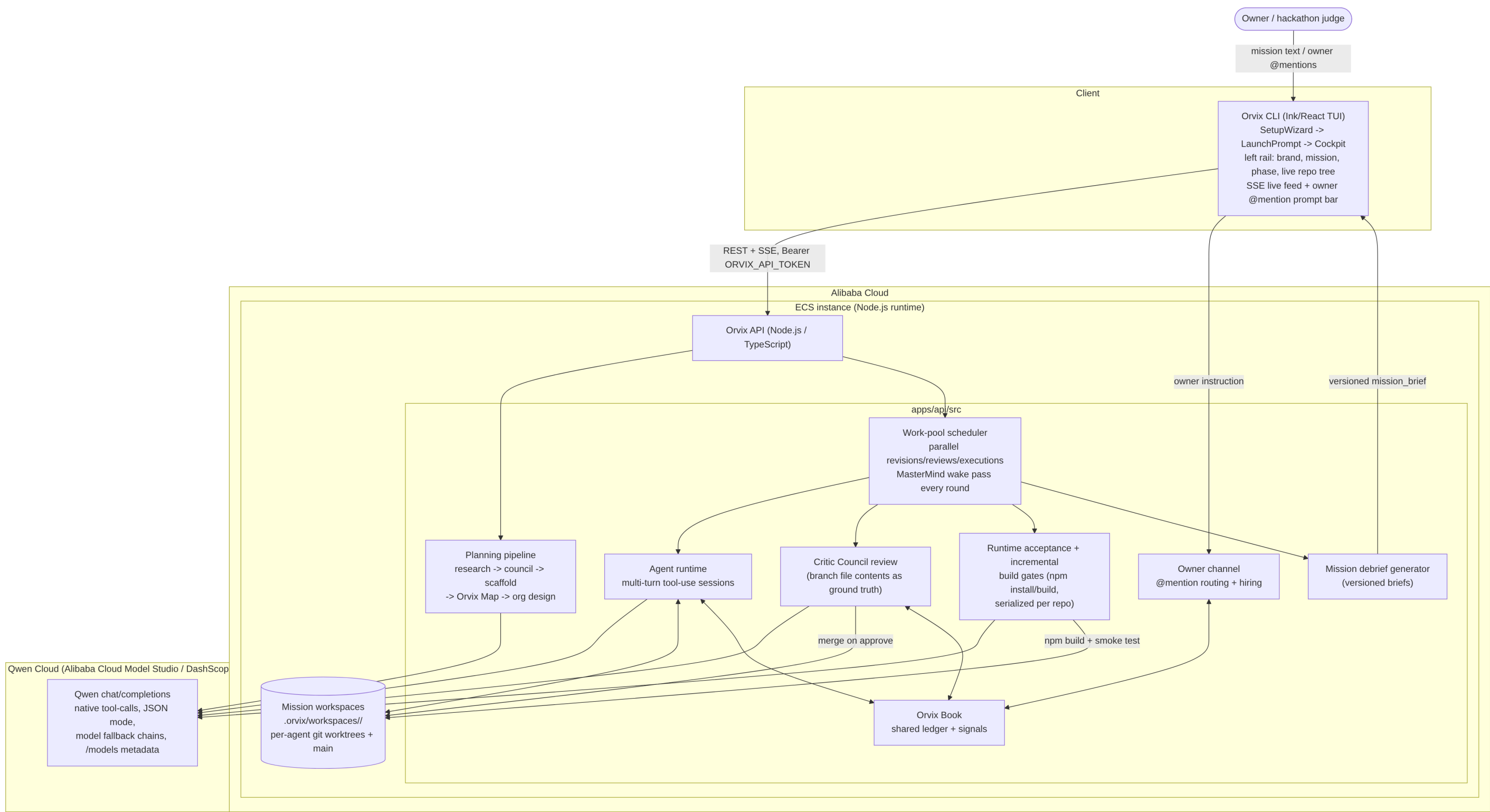
Runs live on **Alibaba Cloud** (ECS + Docker) · models served through **Qwen Cloud** (Alibaba Cloud Model Studio / DashScope)

Frontend: Orvix CLI (Ink/React terminal cockpit) → Backend: Orvix API (Node.js/TypeScript) on Alibaba Cloud ECS → Model layer: Qwen via Qwen Cloud (Alibaba Cloud Model Studio)



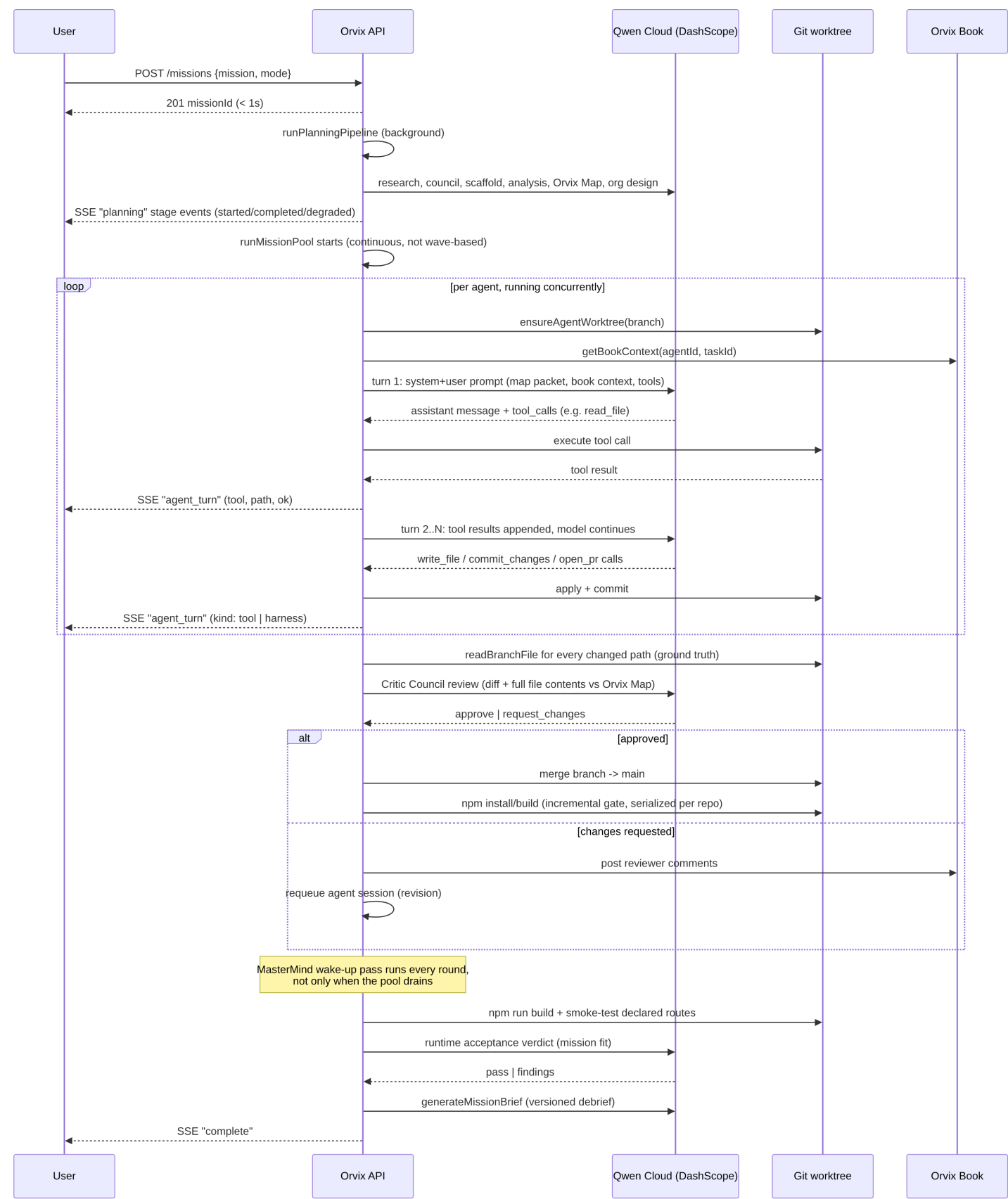
System overview

Client (CLI) → Orvix API on Alibaba Cloud ECS → Qwen Cloud (Alibaba Cloud Model Studio) — with a per-mission git workspace standing in for a database.



Mission lifecycle, end to end

From one sentence to a shipped, reviewed, runtime-verified product — every Qwen call shown against Qwen Cloud (DashScope).



What each piece is, and why it's there

Frontend — Orvix CLI

An Ink/React terminal cockpit. Talks to the API only over REST + Server-Sent Events — it holds no Qwen credentials and touches no git state directly, so it can run on any machine while the mission runs entirely on Alibaba Cloud.

Backend — Orvix API, on Alibaba Cloud ECS

A single Node.js/TypeScript process (Docker container) holding mission state, the scheduler, agent runtime, review, and acceptance gates. No queue, no separate database — state is a `MissionRun` object plus continuous on-disk snapshots.

Model layer — Qwen Cloud

Every planning, agent, review, and acceptance call goes through **Qwen Cloud**'s OpenAI-compatible endpoint at `dashscope-intl.aliyuncs.com` (Alibaba Cloud Model Studio / DashScope).

"Database" — Mission git workspace

Each mission gets its own git repo under `.orvix/workspaces/<missionId>/`, one worktree per agent branch, plus append-only run snapshots under `.orvix/runs/<missionId>/` — this is what survives a process restart.